

프로그래밍 언어 명세와 이모저모 (1/3)

이번 글에서는 프로그래밍 언어를 정의한다는 것은 무슨 의미인지, 그리고 현재 프로그래밍 언어는 어떻게 정의되고 있는지 논의한다. 다음 글에서는 정의의 대상이 되는 ‘프로그래밍 언어’에 대하여 더 자세히 설명하고, 마지막 글에서는 이번 서베이에서 이를 기반으로 무엇을 조사해야 하는지 정리한다.

우리는 프로그래밍 언어를 정의하는 무언가를 프로그래밍 언어의 명세라고 부른다. 프로그래밍 언어의 명세가 존재한다면 우리는 그것으로부터 어떤 표현이 유효한지, 그리고 유효한 표현은 어떤 의미를 가지는지 논의할 수 있어야 할 것이다. 즉 명세는 프로그래밍 언어로 작성할 수 있는 모든 표현의 집합 P 에 대하여 유효한 표현의 집합 $P_{wf} \subseteq P$, 그리고 임의의 $P_{target} \subseteq P_{wf}$ 과 가능한 모든 의미를 포함한 집합 M 에 대하여 $f: P_{target} \rightarrow M$ 을 표현하는 것이다.

단순히 가능한 모든 경우를 나열하는 방식으로 명세를 기술하는 것은 충분하지 않다. 추상적으로 P 는 $\mathbb{Q}, \mathbb{R}, \mathbb{Z}, \mathbb{N}, \mathbb{B}, \mathbb{C}$ 와 문자 집합을 재귀적으로 조합해 정의한 임의의 집합¹으로 표현할 수 있을 것이다. 이렇게 정의하게 되면 P 는 적어도 무수히 많거나, 많은 경우 실제로 무한하다. 따라서 예제를 나열하는 방식으로는, 오직 유한한 (충분하지 못한) 크기의 P_{target} 에 대해서만 의미를 정의할 수 있다. 전체 P_{wf} 를 표현하지 못함도 자명하다.

¹ BNF 표기법을 사용하자면, $P ::= S \mid P \times P$ where $S \subseteq \mathbb{Q}, \mathbb{R}, \mathbb{Z}, \mathbb{N}, \mathbb{B}, \mathbb{C}, \{a, b, c, \dots\}$ 과 같다.

이에 많은 프로그래밍 언어의 명세는 크게 다음 두 가지 방식을 통해 P_{wf} 와 f 를 정의하는 것으로 보인다. 이는 엄밀한 분류라기보단, 사람들이 접근하는 멘탈 모델로 서로 혼합되어 사용되곤 한다.

첫 번째는 P_{wf} 와 f 가 만족해야 하는 성질이나 조건을 제시하여 정의하는 방식이다. 이 방식에서는 P_{wf} 와 f 등 각 집합 사이의 관계를 논리식과 추론 규칙을 통해 정의된다. 따라서 명세에 포함된 각 주장이나 성질이 어떤 전제에서 출발하여 어떻게 도출되는지 명확하게 확인할 수 있으며, 명세와 관련하여 논증하기 편하다는 장점이 있다.

두 번째는 계산 모델²을 이용하여 기술하는 것이다. 임의의 $x \in P$ 에 대하여 $x \in P_{wf}$ 인지 아닌지 판단하는 C_1 과, 임의의 $x \in P_{target}$ 에 대하여 $f(x)$ 를 계산하는 C_2 를 설명/제공하고, 실제로 사용자가 그 계산 모델을 구현/실행하도록 할 수 있도록 하여 P_{wf} 와 f 를 표현하는 방식이다. 이렇게 명세를 작성하게 되면, 명세가 실제로 실행 가능하거나 명세를 기초로 하여 구현체를 만들기 편하다는 장점이 있다.

형식화는 앞선 두 예시처럼 명확하게 정의된 표현 체계를 명시적으로 드러내어 명세를 기술하는 방식이다. 자연어로 기술된 명세에서는 자연어 특유의 모호함뿐 아니라, 명세가 어떤 방식으로 대상을 정의하고 있는지가 명확하게 드러나지 않는 경우도 있다. 실제 명세에서는 앞서 설명한 두 가지 기술 방식을 함께 사용하는 경우도 있다. 예를 들어 구체적인 계산 과정을

² 계산 모델은 계산을 이론적으로 설명하기 위해 제시된 개념으로, 튜링 머신이나 재작성 시스템 등이 대표적이다. 계산 모델에는 다양한 예시가 존재하지만, 튜링 완전성을 고려하여 이번 글에서는 튜링 기계를 간략화한 임의의 계산 모델만을 고려한다. 이는 상태($s \in S$)에 대해서 상태를 전환하는 규칙들($\rightarrow \subseteq S \times S$)로 구성되고, 글에서는 계산 모델의 약어로 $C = \langle S, \rightarrow \rangle$ 을 사용한다.

규정할 필요가 없는 부분에서는 조건이나 성질을 제시하는 방식이 사용되거나, 성질을 통해 특정 계산 과정이 언어의 의미를 결정하는 부분에서는 구체적인 연산 방식을 제시할 수 있다. 이러한 혼합 자체가 항상 문제가 되는 것은 아니지만, 각 기술 방식이 무엇을 정의하며 서로 어떻게 결합되는지가 명확하지 않은 경우 명세의 작성, 유지보수 및 이해를 어렵게 할 수 있다. 형식화는 이러한 정의에 사용되는 개념, 관계 및 계산 구조를 명시적인 형식으로 표현함으로써 명세의 구조와 그에 내재된 가정을 보다 명확하게 드러낼 수 있다.